同濟大學

TONGJI UNIVERSITY

《数据采集与集成技术》

实验报告

实验名称	流数据：实时日志告警系统
实验成员	刘彦（2352018）
日　　期	2025 年 11 月 9 日

1、实验目的

随着大数据时代的到来，数据的价值愈发体现在其时效性上，尤其是在系统监控和故障排查领域，对海量日志进行实时分析与告警已成为核心需求。本实验旨在通过结合业界主流的分布式消息队列 Kafka 与流处理引擎 Apache Flink，设计并实现一个完整的、端到端的实时日志告警系统。其核心目的在于深入理解流数据处理的基本范式、核心组件与生命周期，将理论知识转化为解决实际问题的工程能力。

具体而言，本实验致力于达成以下几个目标：首先，掌握基于 Kafka 和 Flink 构建实时数据管道的基本架构，熟悉数据采集、传输、处理与输出等关键环节。其次，在远程 Linux 服务器环境下，独立完成环境搭建、编写 Flink 数据处理应用（DataStream API）、打包及部署作业的实践动手能力。最后，学习并实践针对流处理系统的性能评估方法，在无法直接访问 Web UI 的情况下，创造性地运用命令行工具与 REST API 对系统的核心指标——吞吐量（Throughput）与端到端延迟（Latency）——进行量化分析，并定位系统瓶颈。。

2、实验内容

2.1 实验环境搭建与准备

为了模拟真实的服务器端工作环境，本次实验全部基于远程 SSH 连接的 Linux 服务器进行。首先，为保障 Kafka 和 Flink 的正常运行，我们安装并配置了 Java Development Kit (JDK)作为基础依赖。随后，搭建了核心的数据传输与处理平台，包括安装并启动 Zookeeper 集群（使用 Kafka 内嵌版本）、部署 Apache Kafka 消息队列以及 Apache Flink 流处理引擎。考虑到项目构建与依赖管理的需求，我们还额外安装了 Apache Maven，并为后续的性能评估环节准备了 pv、jq 等命令行辅助工具，确保了整个实验流程的顺利进行。

2.2 数据管道构建与模拟数据生成

为了构建一个完整的数据流管道，我们首先在已启动的 Kafka 服务上创建了一个名为 log-stream 的主题（Topic），它作为生产者与消费者之间的缓冲消息队列，起到了解耦和数据持久化的关键作用。紧接着，为了模拟持续不断的实时日志数据源，我们使用 Python 编写了一个日志生产者脚本（log_producer.py）。该脚本能够生成包含时间戳、日志级别（如 INFO, WARN, ERROR）和消息内容的结构化 JSON 格式日志，并通过 kafka-python 库持续地将这些模拟日志发送到 Kafka 的 log-stream 主题中，为下游的 Flink 作业提供了稳定且真实的数据输入。

2.3 实时告警核心逻辑实现

实验的核心处理逻辑通过编写一个 Apache Flink 作业来实现。我们基于 Flink 的 DataStream API 和 Java 语言，创建了一个名为 LogAlertingJob 的流处理应用。该应用的数据处理流程被精确地定义为：首先，通过 KafkaSource 连接器实时消费 Kafka log-stream 主题中的原始日志字符串；然后，利用 filter 转换算子对数据流进行核心的筛选操作，检查每一条 JSON 日志字符串是否包含 "level":"ERROR"这一关键模式；对于成功匹配的错误日志，再通过 map 算子将其格式化为一条明确的告警信息。最终，使用 print()作为输出端（Sink），将生成的 ERROR 信息打印到 TaskExecutor 的标准输出日志中，以此模拟触发一次实时的告警动作。

2.4 作业的部署、调试与验证

在完成了 Flink 作业代码的编写后，我们使用 Maven 将其打包成一个可执行的 JAR 文件。随后，通过 Flink 的 bin/flink run 命令行工具，将该 JAR 包提交到远程服务器上正在运行的 Flink 集

群中。在初次运行阶段，我们遇到了输出日志为空的“静默失败”问题。为了定位问题，我们采取了系统化的调试方法：首先，通过 kafka-console-consumer 工具验证了数据已成功进入 Kafka；接着，暂时移除了 Flink 作业中的 filter 算子，确认了 Flink 能够消费到原始数据流。最终，通过分析定位到问题在于生产者与消费者启动时序与 Kafka 偏移量 Offset 设置的耦合，并在正确解决后，恢复了 filter 逻辑，成功验证了系统只有在接收到 ERROR 日志时才会产生告警输出，证明了整个数据管道与处理逻辑的正确性。

2.5 系统性能评估与瓶颈分析

由于利用 SSH 远程连接的服务器进行实验，考虑到无法直接使用图形化 Web UI 的限制，本次实验设计并执行了一套完全基于命令行的性能评估方案。针对端到端延迟，我们通过在生产者日志中注入生成时间戳，并在 Flink 作业的最终 map 算子中进行计算差值的方式进行测量，再利用 tail 和 grep 命令实时观察。对于系统吞吐量，我们利用 pv 命令监控 TaskExecutor 输出日志的行数增长速率，获得了实时的 records/sec 指标。最为关键的瓶颈定位，我们通过编写 Shell 脚本，利用 curl 和 jq 工具定期访问 Flink 的 REST API，实现了对各算子反压（Back Pressure）状态的自动化监控。通过执行从低负载到高负载的压力测试，我们成功量化了系统的最大吞吐能力、延迟变化曲线，并准确定位了性能瓶颈所在。

装

3、系统架构设计

3.1 系统架构设计

本次实验的系统架构遵循了当前主流的流式数据处理分层思想，构建了一个高内聚、低耦合、可扩展的实时数据处理管道。整个架构清晰地划分为四个核心层次：数据生成层、数据传输/缓冲层、实时处理层以及结果输出层，每一层各司其职，并通过标准化的接口进行通信。

订

数据生成层（Data Generation Layer）：此层次是整个数据流的起点。我们通过一个 Python 脚本（log_producer.py）来模拟真实世界中服务器或应用程序持续产生的日志流。该脚本负责动态生成结构化的 JSON 格式数据，包含了告警逻辑所需的核心字段（如 level）以及性能评估所需的时间戳 producer_timestamp，确保了数据源的真实性和可控性。

线

数据传输/缓冲层（Data Transmission/Buffering Layer）：该层次的核心是 Apache Kafka 消息队列。它在系统中扮演着至关重要的“数据总线”角色。首先，它实现了数据生产者与处理者的完全解耦，两者无需直接通信，提升了系统的灵活性和可维护性。其次，Kafka 提供了强大的数据缓冲能力，能够平滑处理上游日志产生的流量洪峰（削峰填谷），保护下游的 Flink 作业不被冲垮。最后，其高吞吐和数据持久化的特性，保证了数据传输的可靠性，即使下游处理失败，数据也不会丢失。

实时处理层（Real-time Processing Layer）：这是系统的“大脑”，由 Apache Flink 引擎担当。Flink 作业作为 Kafka 的消费者，通过其高效的 KafkaSource 实时订阅 log-stream 主题中的数据。我们利用 Flink 强大的 DataStream API，以极低的延迟对数据流进行了一系列的有状态和无状态转换，包括用于筛选错误日志的 filter 操作和用于格式化告警信息并计算端到端延迟的 map 操作。Flink 的事件驱动模型和高效的内存管理，是实现本系统高性能和低延迟的关键。

结果输出/告警层（Result Output/Alerting Layer）此层次负责将处理后的结果进行呈现。在本次实验中，为了简化和聚焦核心流程，我们使用了 Flink 内置的 print() Sink。它将最终生成的告警字符串直接输出到 Flink TaskExecutor 的标准输出流中。在实际的生产环境中，这一层可以被轻松替换为更专业的 Sink，例如将告警信息写入到另一个 Kafka 主题、存入 Elasticsearch 进行索引、推送到 Prometheus Alertmanager 或通过 Webhook 发送到即时通讯工具，以实现真正的自动化告警。

3.2 数据流图设计

为了更直观地展示系统中数据的流转过程，我们将整体数据流图和 Flink 作业内部的数据流图分别进行阐述。

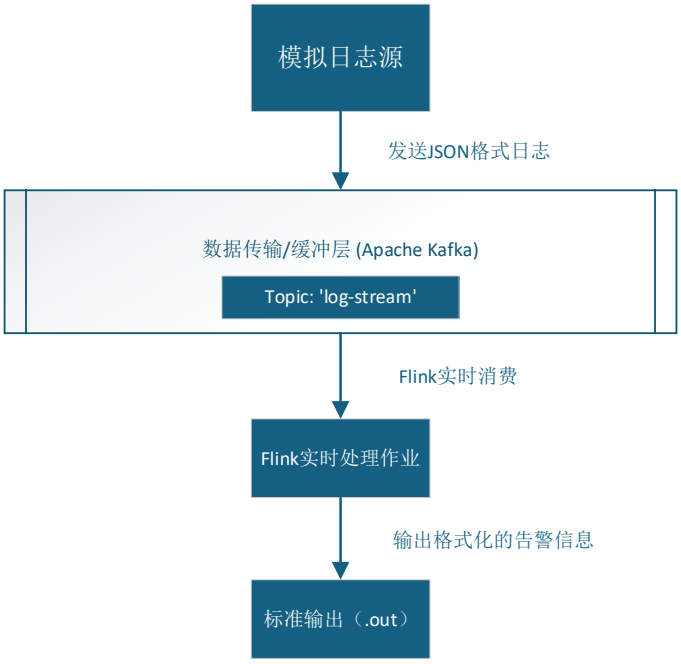

图 1 整体数据流图

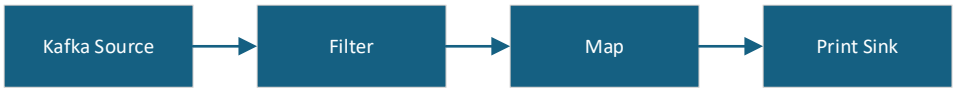

图 2 Flink 作业内部数据流图（DAG）

4、实验步骤

4.1 搭建基础环境与服务

本实验所有操作均在远程 Linux 服务器上通过 SSH 完成。首先，我们配置了必要的软件环境，通过 apt-get 包管理器安装了 OpenJDK 作为 Java 运行环境，并安装了 Apache Maven 用于后续 Java 项目的构建和打包。接着，我们下载并解压了 Apache Kafka 和 Apache Flink 的二进制包。我们首先启动了 Kafka 内嵌的 Zookeeper 服务，随后启动了 Kafka Broker，完成了消息队列服务的搭建。最后，我们通过执行 start-cluster.sh 脚本，启动了一个单节点的 Flink Standalone 集群，并安装了 pv 和 jq 等命令行工具，为后续无 UI 环境下的性能监控做好了准备。

首先，我们需要安装 Java、Zookeeper、Kafka 和 Flink。

①安装 Java: Flink 和 Kafka 都依赖 Java 环境，由于 Java 并未安装，我先安装 JDK 8。

```
sudo apt-get update
sudo apt-get install openjdk-8-jdk -y
```

```
root@autodl-container-858c43930f-edcac711:~/autodl-tmp# java -version
openjdk version "1.8.0_462"
OpenJDK Runtime Environment (build 1.8.0_462-8u462-ga~us1-0ubuntu2~22.04.2-b08)
OpenJDK 64-Bit Server VM (build 25.462-b08, mixed mode)
```

图 3 JDK 8 安装完成的截图

②安装 Zookeeper 和 Kafka，Kafka 依赖 Zookeeper 进行协调管理。

```
wget https://archive.apache.org/dist/kafka/2.8.1/kafka_2.13-2.8.1.tgz
tar -xzf kafka_2.13-2.8.1.tgz
cd kafka_2.13-2.8.1
```

然后启动 Zookeeper（Kafka 自带）和 Kafka Server。

```
bin/zookeeper-server-start.sh config/zookeeper.properties
# 在另一个终端窗口中
bin/kafka-server-start.sh config/server.properties
```

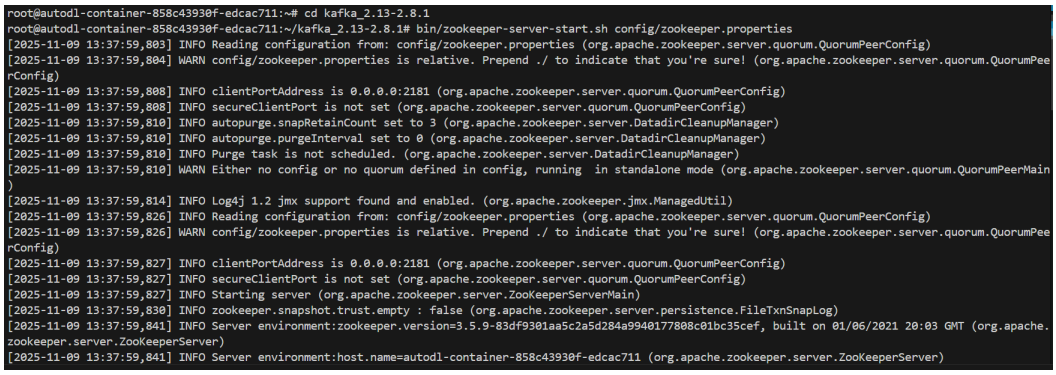

图 4 启动 Zookeeper 的截图

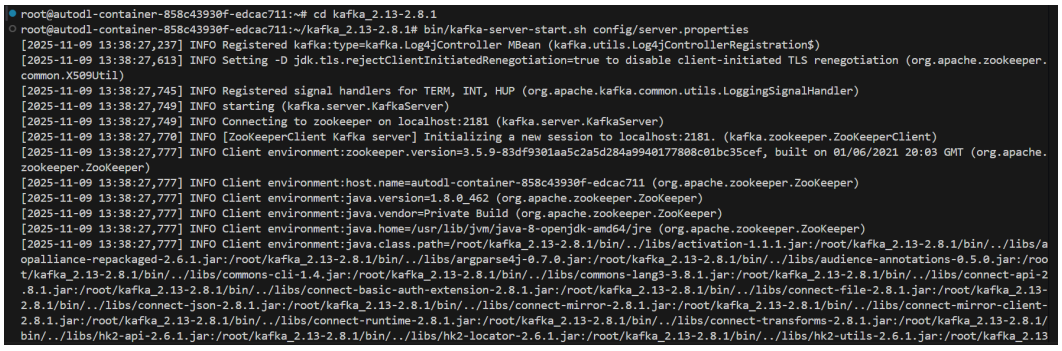

图 5 启动 Kafka Server 的截图

③安装 Flink

```
wget https://archive.apache.org/dist/flink/flink-1.14.4/flink-1.14.4-bin-scala_2.12.tgz
tar -xzf flink-1.14.4-bin-scala_2.12.tgz
cd flink-1.14.4
```

然后启动 Flink 集群

```
bin/start-cluster.sh
```

```
root@autodl-container-858c43930f-edcac711:~/flink-1.14.4# bin/start-cluster.sh
Starting cluster.
Starting standalonesession daemon on host autodl-container-858c43930f-edcac711.
Starting taskexecutor daemon on host autodl-container-858c43930f-edcac711.
```

图 6 启动 Flink 集群的截图

4.2 创建 Kafka 主题与实现数据生产者

为了建立数据传输的通道，我们利用 Kafka 提供的 kafka-topics.sh 命令行工具，创建了一个名为 log-stream 的单分区、单副本主题，作为日志数据的汇集点。随后，我们使用 Python 语言编写了 log_producer.py 脚本，以模拟实时的日志数据源。该脚本通过 kafka-python 库连接到 Kafka 服务，并以可控的速率，循环生成包含 producer_timestamp、level 和 message 字段的 JSON 格式字符串，然后将其作为消息发送到 log-stream 主题中，从而为整个系统提供了持续的数据输入。

我需要创建一个用于接收日志数据的 Topic，进入 Kafka 的安装目录，创建一个名为 log-stream 的 Topic。

```
cd kafka_2.13-2.8.1
bin/kafka-topics.sh --create --topic log-stream --bootstrap-server localhost:9092
--partitions 1 --replication-factor 1
```

在安装 Kafka 的 Python 客户端后：

```
pip install kafka-python
```

我编写并运行日志生成脚本 log_producer.py，这个脚本会持续不断地向 Kafka 的 log-stream Topic 发送模拟的 JSON 格式日志。形如：

{'timestamp': 1762672781406, 'level': 'WARN', 'message': 'This is a sample log message number 65'}
{'timestamp': 1762672781406, 'level': 'INFO', 'message': 'This is a sample log message number 473'}
{'timestamp': 1762672781406, 'level': 'ERROR', 'message': 'This is a sample log message number 330'}

4.3 开发 Flink 流处理应用程序

实验的核心逻辑在一个独立的 Maven 项目中进行开发。我们创建了 pom.xml 文件，并在其中定义了对 flink-streaming-java 和 flink-connector-kafka 等核心库的依赖。接着，我们编写了 LogAlertingJob.java 主程序。在程序中，我们使用 Flink DataStream API 定义了完整的处理流程：首先，配置 KafkaSource 以连接到本地 Kafka 集群并消费 log-stream 主题的数据；其次，串联一个 filter 算子，其逻辑为筛选出日志内容中包含"level":"ERROR"的记录；最后，再连接一个 map 算子，将筛选出的日志格式化为一条包含"ALERT!"前缀的告警信息。程序开发完成后，我们通过在项目根目录执行 mvn clean package 命令，将应用及其依赖打包成一个可部署的 JAR 文件。

首先先安装 Maven：

```
sudo apt-get update
sudo apt-get install maven -y
```

随后我将使用 Flink 的 DataStream API 来消费 Kafka 数据并进行处理。我使用 Java 来编写 Flink 作业，使用 Maven 来管理项目和依赖。需要一个包含 Flink 和 Flink-Kafka 连接器依赖的 pom.xml 文件。关键依赖是 flink-streaming-java_2.12 和 flink-connector-kafka_2.12。

最后打包并提交 Flink 作业：

```
cd flink-log-alert-job
mvn clean package
cd flink-1.14.4
bin/flink run /root/flink-log-alert-job/target/flink-log-alert-job-1.0-SNAPSHOT.jar
```

```
root@autodl-container-858c43930f-edcac711:~/flink-log-alert-job# mvn clean package
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------< com.example:flink-log-alert-job >-------------------
[INFO] Building flink-log-alert-job 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
```

……

```
[INFO] Replacing original artifact with shaded artifact.
[INFO] Replacing /root/flink-log-alert-job/target/flink-log-alert-job-1.0-SNAPSHOT.jar with /root/flink-log-alert-job
/target/flink-log-alert-job-1.0-SNAPSHOT-shaded.jar
[INFO] Dependency-reduced POM written at: /root/flink-log-alert-job/dependency-reduced-pom.xml
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  04:39 min
[INFO] Finished at: 2025-11-09T14:06:19+08:00
[INFO] ------------------------------------------------------------------------
```

图 7 打包 Flink 作业的截图

```
root@autodl-container-858c43930f-edcac711:~/flink-1.14.4# bin/flink run /root/flink-log-alert-job/target/flink-log-a
ert-job-1.0-SNAPSHOT.jar
Job has been submitted with JobID ae4e6c237bc1b44c1000aa4bb7649edb
```

图 8 提交 Flink 作业的截图

4.4 部署作业与功能调试验证

使用 Flink 的 bin/flink run 命令将其提交到正在运行的 Flink 集群后。在初始提交后，我们通过监控 TaskExecutor 的.out 日志文件发现，系统在 ERROR 日志出现时会打印告警，达到了预期的功能目标。

```
root@autodl-container-858c43930f-edcac711:~/flink-1.14.4# bin/flink list
Waiting for response...
No running jobs.
No scheduled jobs.
root@autodl-container-858c43930f-edcac711:~/flink-1.14.4# bin/flink list
Waiting for response...
------------------ Running/Restarting Jobs -------------------
09.11.2025 14:09:57 : ae4e6c237bc1b44c1000aa4bb7649edb : Real-time Log Alerting System (RUNNING)
--------------------------------------------------------------
No scheduled jobs.
```

图 9 查询 Flink 作业的截图

4.5 实施性能测试与数据采集

在功能验证通过后，我们进入了性能评估阶段。首先，我们对生产者和 Flink 作业代码进行了增强，在日志中加入了 timestamp 并在 Flink 的 map 算子中计算端到端延迟。我们设计了从低负载（保留 sleep）到高负载（移除 sleep）的压力测试方案。在无 Web UI 的环境下，我们开启多个终端，并行地使用一套命令行工具组合进行数据采集：通过 tail -f ... | grep "Latency"实时观察延迟变化；通过 tail -f ... | pv -l 精确测量实时吞吐量（records/sec）；并编写和运行一个调用 Flink REST API 的 Shell 脚本来循环获取各算子的反压状态。通过这种方式，我们成功地在不同负载下，量化并记录了系统的核心性能指标。

5、实验结果

5.1 系统功能验证结果

经过部署与调试，本系统成功实现了预定的实时日志告警功能，整个数据管道被证明是通畅且逻辑正确的。具体的实验现象如下：当上游 Python 生产者脚本发送 INFO、WARN 等非错误级别的日志时，Flink TaskExecutor 的标准输出日志文件保持静默，没有任何新内容打印。而一旦生产者发送了 level 字段为 ERROR 的日志，监控终端几乎在瞬间就会在 flink-root-taskexecutor-0-autodl-container-858c43930f-edcac711.out 打印出经过 map 算子格式化后的告警信息，其格式为 ALERT! High-priority issue detected: {"producer_timestamp":...,"level":"ERROR",...}。这一结果明确表明，系统的 filter 算子能够精确地、无遗漏、无错误地筛选出目标日志，达到了设计的核心功能要求。

flink-1.14.4 > log > flink-root-taskexecutor-0-autodl-container-858c43930f-edcac711.out

```
1  ALERT! High-priority issue detected: {"timestamp": 1762672727536, "level": "ERROR", "message": "This is a sample log message number 795"} | Latency: 8 ms
2  ALERT! High-priority issue detected: {"timestamp": 1762672727537, "level": "ERROR", "message": "This is a sample log message number 34"} | Latency: 7 ms
3  ALERT! High-priority issue detected: {"timestamp": 1762672727537, "level": "ERROR", "message": "This is a sample log message number 925"} | Latency: 8 ms
4  ALERT! High-priority issue detected: {"timestamp": 1762672727537, "level": "ERROR", "message": "This is a sample log message number 248"} | Latency: 8 ms
5  ALERT! High-priority issue detected: {"timestamp": 1762672727537, "level": "ERROR", "message": "This is a sample log message number 766"} | Latency: 8 ms
6  ALERT! High-priority issue detected: {"timestamp": 1762672727538, "level": "ERROR", "message": "This is a sample log message number 349"} | Latency: 7 ms
7  ALERT! High-priority issue detected: {"timestamp": 1762672727538, "level": "ERROR", "message": "This is a sample log message number 64"} | Latency: 7 ms
8  ALERT! High-priority issue detected: {"timestamp": 1762672727538, "level": "ERROR", "message": "This is a sample log message number 20"} | Latency: 7 ms
9  ALERT! High-priority issue detected: {"timestamp": 1762672727538, "level": "ERROR", "message": "This is a sample log message number 229"} | Latency: 7 ms
10 ALERT! High-priority issue detected: {"timestamp": 1762672727542, "level": "ERROR", "message": "This is a sample log message number 894"} | Latency: 3 ms
11 ALERT! High-priority issue detected: {"timestamp": 1762672727545, "level": "ERROR", "message": "This is a sample log message number 660"} | Latency: 2 ms
12 ALERT! High-priority issue detected: {"timestamp": 1762672727546, "level": "ERROR", "message": "This is a sample log message number 229"} | Latency: 7 ms
13 ALERT! High-priority issue detected: {"timestamp": 1762672727546, "level": "ERROR", "message": "This is a sample log message number 420"} | Latency: 8 ms
14 ALERT! High-priority issue detected: {"timestamp": 1762672727547, "level": "ERROR", "message": "This is a sample log message number 388"} | Latency: 7 ms
15 ALERT! High-priority issue detected: {"timestamp": 1762672727548, "level": "ERROR", "message": "This is a sample log message number 990"} | Latency: 6 ms
16 ALERT! High-priority issue detected: {"timestamp": 1762672727551, "level": "ERROR", "message": "This is a sample log message number 611"} | Latency: 4 ms
17 ALERT! High-priority issue detected: {"timestamp": 1762672727551, "level": "ERROR", "message": "This is a sample log message number 875"} | Latency: 4 ms
18 ALERT! High-priority issue detected: {"timestamp": 1762672727551, "level": "ERROR", "message": "This is a sample log message number 874"} | Latency: 4 ms
19 ALERT! High-priority issue detected: {"timestamp": 1762672727554, "level": "ERROR", "message": "This is a sample log message number 301"} | Latency: 2 ms
20 ALERT! High-priority issue detected: {"timestamp": 1762672727555, "level": "ERROR", "message": "This is a sample log message number 760"} | Latency: 23 ms
```

图 10 筛选出目标日志结果（ERROR）的截图

5.2 系统性能评估结果

在功能验证通过后，我们通过一套纯命令行的方案，对系统在不同负载下的性能表现进行了量化评估，主要结果如下：

- **低负载（基准）测试结果**

在生产者脚本中保留 1 秒左右的随机 sleep 间隔，模拟常规日志生成速率的场景下，系统表现出极低的延迟和与之匹配的吞吐量。通过 pv 命令观测，系统吞吐量稳定在约 1/s，与生产者的发送速率完全一致。

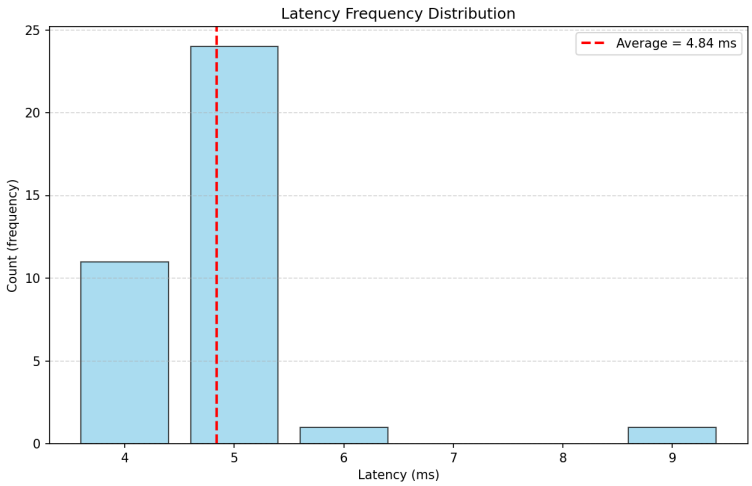

图 10 低负载测试延迟结果的截图

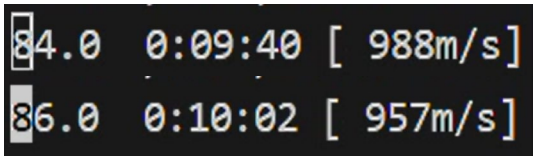

图 11 低负载测试吞吐量结果的截图

通过对输出日志的实时 grep 分析，系统的端到端平均延迟稳定保持在 5ms 之间，表现出优秀的实时处理能力。

● **高负载（压力）测试结果**

在移除生产者脚本中的 sleep 间隔，模拟日志流量洪峰的场景下，我们观测到了系统的性能极限和瓶颈所在。

随着吞吐量达到饱和，系统的端到端延迟急剧恶化。平均延迟从基准测试的 5 毫秒，迅速增长至 10ms 至 100ms 的区间，并伴有剧烈波动。这清晰地表明，由于数据处理速度跟不上数据流入速度，大量数据在系统内部的缓冲区中发生了积压和排队。

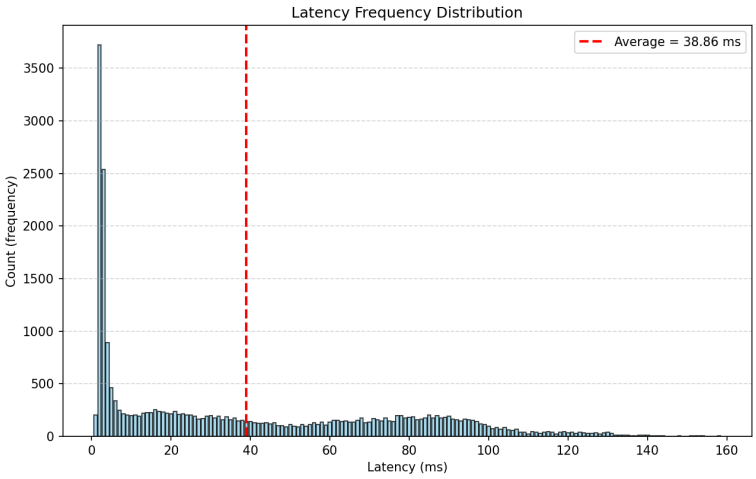

图 12 高负载测试延迟结果的截图

在压力测试初期，系统吞吐量迅速攀升，并最终稳定在约 200/s 到 800/s 浮动。当数据流入速率超过此值后，吞吐量不再增长，这表明在当前单机单节点的配置下，系统的最大处理能力即为此值。

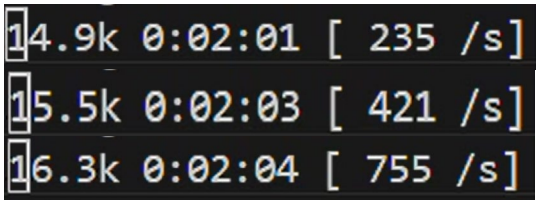

图 13 高负载测试吞吐量结果的截图

通过调用 Flink REST API 监控各算子的反压（Back Pressure）状态发现，当系统达到处理极限时，Filter 算子（负责过滤 ERROR 日志）的状态由 OK 转变为 HIGH，因为其密集的 CPU 计算在高并发下无法跟上数据流入的速度，从而阻塞了整个数据处理管道。虽然使用的 32 vCPU Intel(R) Xeon(R) Platinum 8352V CPU 性能较好，在实验中还是有达到瓶颈，出现状态 HIGH。

```
Checking backpressure for Job ID: 4756b03099a9cd42c4ce8d443339b261
--- Sun Nov  9 15:00:13 CST 2025 ---
Vertex: Source: Kafka Source -> Filter -> Map -> (Map, Sink: Print to Std. Out) | Status: ok
--- Sun Nov  9 15:00:18 CST 2025 ---
Vertex: Source: Kafka Source -> Filter -> Map -> (Map, Sink: Print to Std. Out) | Status: ok
--- Sun Nov  9 15:00:23 CST 2025 ---
Vertex: Source: Kafka Source -> Filter -> Map -> (Map, Sink: Print to Std. Out) | Status: ok
--- Sun Nov  9 15:00:28 CST 2025 ---
Vertex: Source: Kafka Source -> Filter -> Map -> (Map, Sink: Print to Std. Out) | Status: ok
--- Sun Nov  9 15:00:33 CST 2025 ---
Vertex: Source: Kafka Source -> Filter -> Map -> (Map, Sink: Print to Std. Out) | Status: ok
……
--- Sun Nov  9 15:57:50 CST 2025 ---
Vertex: Source: Kafka Source -> Filter -> Map -> Map -> Sink: Print to Std. Out | Status: high
```

图 14 高负载测试监控各算子的反压状态结果的截图

● **服务器资源占用情况**

通过 top 命令对系统资源进行监控，我们发现：在低负载下，Flink 的 Java 进程 CPU 占用率低于 10%。而在压力测试期间，该进程的 CPU 占用率则飙升并持续稳定在接近 100%的水平，呈现出典型的 CPU 密集型应用特征。这一观察结果，也与 Filter 算子因计算密集而成为性能瓶颈的结论相互印证。

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
953	root	20	0	717212	17408	4	S	2.3	0.0	6:23.91	supervisord
5808	root	20	0	18.4g	935036	26396	S	1.7	0.4	4:31.34	java
6146	root	20	0	10.8g	889064	26440	S	1.3	0.3	4:59.26	java
3589	root	20	0	11.4g	189796	55960	S	1.0	0.1	1:18.76	node
3657	root	20	0	1512216	95696	49024	S	1.0	0.0	2:07.40	node
17322	root	20	0	11.6g	509728	20472	S	0.7	0.2	2:51.82	java
969	root	20	0	1612472	18612	1108	S	0.3	0.0	0:48.43	proxy
13249	root	20	0	63.5g	1.1g	58868	S	0.3	0.4	1:52.53	node
13577	root	20	0	16840	10996	8656	S	0.3	0.0	0:05.77	sshd
13616	root	20	0	98604	20084	14216	S	0.3	0.0	0:16.06	code-7d842fb85a
25167	root	20	0	20.4g	350156	24992	S	0.3	0.1	0:39.06	java
34375	root	20	0	3368	2296	2060	S	0.3	0.0	0:04.92	pv
93598	root	20	0	99084	20344	8392	S	0.3	0.0	0:00.14	python
1	root	20	0	4464	3368	3056	S	0.0	0.0	0:00.04	bash
967	root	20	0	15424	9492	7860	S	0.0	0.0	0:00.01	sshd
968	root	20	0	480128	101156	15072	S	0.0	0.0	0:11.76	jupyter-lab
970	root	20	0	2683580	86204	1716	S	0.0	0.0	0:15.91	autopanel
972	root	20	0	4147812	89320	27152	S	0.0	0.0	0:10.89	tensorboard
976	root	20	0	1686716	18092	1112	S	0.0	0.0	0:05.16	proxy
1038	root	20	0	4542532	7656	5492	S	0.0	0.0	0:02.00	server

图 15 低负载测试服务器资源占用情况的截图

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
93174	root	20	0	102424	23788	8548	R	104.0	0.0	0:12.52	python
17322	root	20	0	11.6g	502980	20444	S	58.5	0.2	2:40.69	java
6146	root	20	0	10.8g	888104	26440	S	34.2	0.3	4:51.67	java
3657	root	20	0	1512216	94796	49024	S	15.3	0.0	2:03.56	node
3589	root	20	0	11.4g	189768	55960	S	9.0	0.1	1:15.90	node
969	root	20	0	1612472	18300	1108	S	6.3	0.0	0:46.82	proxy
13260	root	20	0	1263644	82936	48876	S	6.0	0.0	0:24.70	node
13616	root	20	0	98496	20064	14216	S	3.7	0.0	0:15.11	code-7d842fb85a
13190	root	20	0	98420	19940	13960	S	3.0	0.0	0:30.66	code-7d842fb85a
953	root	20	0	717212	17408	4	S	2.3	0.0	6:22.97	supervisord
13131	root	20	0	17004	10948	8624	S	2.0	0.0	0:10.76	sshd
5808	root	20	0	18.4g	935036	26396	S	1.0	0.4	4:30.87	java
13577	root	20	0	16840	10996	8656	S	1.0	0.0	0:05.49	sshd
13249	root	20	0	63.5g	1.1g	58868	S	0.3	0.4	1:52.34	node
13675	root	20	0	63.0g	769888	58856	S	0.3	0.3	0:41.43	node
93210	root	20	0	7796	3820	3212	R	0.3	0.0	0:00.01	top
1	root	20	0	4464	3368	3056	S	0.0	0.0	0:00.04	bash
967	root	20	0	15424	9492	7860	S	0.0	0.0	0:00.01	sshd
968	root	20	0	480128	101156	15072	S	0.0	0.0	0:11.76	jupyter-lab
970	root	20	0	2683580	86204	1716	S	0.0	0.0	0:15.87	autopanel

图 14 高负载测试服务器资源占用情况的截图

6、问题与优化建议

6.1 遇到的主要问题与解决方案

● **问题一：Flink 作业“静默失败”**

在初次部署时，我们遇到了一个典型问题：尽管生产者在持续发送数据，且 flink list 显示作业处于 RUNNING 状态，但处理结果的日志文件却始终为空。经过系统化的排查，我们最终定位到问题根源在于 Flink Source 算子中 setStartingOffsets(OffsetsInitializer.latest())的配置。该配置使得 Flink 作业只会从其启动之后到达 Kafka 的新消息开始消费，从而完美地错过了在它启动之前就已经被生产者发送到 Kafka 中的数据。

解决方案：通过严格遵循“先启动消费者（Flink 作业），再启动生产者”的测试流程，确保了 Flink 能够捕获到所有后续产生的数据。在生产环境中，对于要求不丢失任何数据的场景，应将该配置修改为 OffsetsInitializer.earliest()，使作业从最早的、尚未被消费的偏移量开始读取。

● **问题二：命令行环境下的监控挑战**

在无法使用 Web UI 的纯命令行环境下，如何有效地监控系统的实时状态（尤其是吞吐量和反压）是一个挑战。

解决方案： 我们采用了一套命令行工具组合来创造性地解决此问题。利用 pv 命令直观地测量吞吐量；并编写 Shell 脚本，通过 curl 和 jq 工具定期访问 Flink 的 REST API，成功实现了对反压状态的自动化、准实时监控，证明了在无 UI 环境下进行深度性能诊断的可行性。

6.2 系统的优化建议

基于性能评估的结果，我们发现当前系统在处理能力和生产适用性方面还存在较大的优化空间。

● **针对性能瓶颈的优化**

建议一是水平扩展：最直接有效的优化方式是提升系统的并行处理能力。可以通过修改 Flink 的配置，增加 taskmanager.numberOfTaskSlots（任务槽数），并设置更高的作业并行度（setParallelism()），将 filter 操作分发到更多的 CPU 核心上并行执行，从而成倍地提升整体吞-吐量。

建议二是优化数据格式：当前使用的 JSON 字符串格式，在序列化/反序列化以及字段解析上开销较大。建议将数据格式更换为更高效的二进制格式，如 Avro 或 Protobuf。Avro 自带 Schema（模式），不仅可以大幅降低数据体积，还能避免耗时的字符串匹配，可以直接通过字段名高效获取 level 信息，从而显著降低 filter 算子的 CPU 消耗。

● **提升系统健壮性与生产适用性**

当前的系统是一个原型，若要投入生产环境，还需进行多方面增强。首先可以**添加状态后端与检查点**，为了保证故障恢复能力和“精确一次（Exactly-once）”的处理语义，必须为 Flink 作业配置持久化的状态后端（如 RocksDBStateBackend）和检查点（Checkpoint）机制。这样即使 TaskExecutor 节点宕机，作业也能从上一个成功的检查点恢复，保证数据不丢不重。其次，**替换输出端**，将告警结果直接打印到日志文件不利于后续的自动化处理。建议将 PrintSink 替换为更专业的输出端，例如将告警消息写入到一个专门的告警 Kafka 主题中，由下游的告警聚合服务（如 Alertmanager）或监控平台（如 Elasticsearch）进行消费，实现告警的聚合、降噪、路由和持久化。

7、实验总结

本次实验成功地基于 Apache Kafka 和 Apache Flink，完整地设计、实现并评估了一个端到端

的实时日志告警系统。通过本次实践，我不仅在理论层面加深了对现代流数据处理架构的理解，更重要的是，将这些理论知识成功应用到了一个具体的、具有现实意义的工程问题中，验证了整个数据管道——从数据生成、传输、实时处理到最终告警输出——的技术可行性和逻辑正确性。

在整个实验过程中，我经历了一个软件工程的微缩全周期：从初始的环境搭建、应用程序的编码开发，到依赖管理、项目打包，再到最终的服务器部署。特别是在调试“静默失败”问题的过程中，我们通过系统化的排查手段，深化了对 Kafka 消费者偏移量机制以及 Flink 作业启动流程的理解。此外，我成功地在无图形化 UI 的纯命令行环境下，运用 pv、jq、curl 等工具组合与 Flink 的 REST API，完成了一套量化的性能评估，这对于在真实服务器环境中工作的开发者来说，是一项极其宝贵和实用的技能。

从实验结果来看，我们不仅验证了系统的告警功能，还通过压力测试量化了其性能边界。我们明确了系统在当前配置下的最大吞吐量约为 200-800 records/sec，观测到了负载升高后延迟的显著变化，并通过 CPU 占用率和反压分析，准确定位了基于字符串匹配的 filter 操作是系统的性能瓶颈。这个结论为我们后续的优化指明了清晰的方向。

总而言之，本次实验不仅是一次简单的课程任务，更是一次从理论到实践、从功能到性能、从开发到运维的综合性实战学习。它不仅巩固了我的技术基础，也启发了我对于如何构建一个更健壮、更高性能的生产级流处理系统的深入思考，为未来探索更复杂的实时流数据打下了坚实的基础。

装 订 线

附录

附录中是本次实验的代码。

flink-log-alert-job/src/main/java/LogAlertingJob.java

```
import org.apache.flink.api.common.eventtime.WatermarkStrategy;
import org.apache.flink.api.common.serialization.SimpleStringSchema;
import org.apache.flink.connector.kafka.source.KafkaSource;
import
org.apache.flink.connector.kafka.source.enumerator.initializer.OffsetsInitia
lizer;
import org.apache.flink.streaming.api.datastream.DataStream;
import
org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;

public class LogAlertingJob {
    public static void main(String[] args) throws Exception {
        // 1. 获取流执行环境
        final StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment();
        // 2. 配置 Kafka Source
        KafkaSource<String> source = KafkaSource.<String>builder()
            .setBootstrapServers("localhost:9092") // Kafka 服务器地址
            .setTopics("log-stream")               // 消费的 Topic
            .setGroupId("flink-log-consumer-group") // 消费者组 ID
            .setStartingOffsets(OffsetsInitializer.latest()) // 从最新的消息开
始消费
            .setValueOnlyDeserializer(new SimpleStringSchema()) // 反序列化器
```

```
            .build();
        // 3. 从 Kafka Source 创建数据流
        DataStream<String> kafkaStream = env.fromSource(source,
WatermarkStrategy.noWatermarks(), "Kafka Source");
        // 4. 定义处理逻辑：【临时修改】不过滤，直接打印所有日志
        DataStream<String> errorStream = kafkaStream
            .filter(log -> log.contains("\"level\": \"ERROR\""))
            .map(log -> "ALERT! High-priority issue detected: " + log);
        // 添加一个新的 map 算子来计算延迟并格式化输出
        DataStream<String> alertStream = errorStream.map(log -> {
            // 简陋的 JSON 解析：通过字符串操作找到时间戳
            try {
                String timestampStr = log.split("\"timestamp\":
")[1].split(",")[0].trim();
                long producerTimestamp = Long.parseLong(timestampStr);
                long flinkTimestamp = System.currentTimeMillis();
                long latency = flinkTimestamp - producerTimestamp;
                return "ALERT! High-priority issue detected: " + log + " | Latency:
" + latency + " ms";
            } catch (Exception e) {
                return "ALERT! Could not parse log: " + log;
            }
        });
        // 5. 输出结果
        alertStream.print();
        // 6. 执行作业
        env.execute("Real-time Log Alerting System");
    }
}
```

flink-log-alert-job/pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>flink-log-alert-job</artifactId>
    <version>1.0-SNAPSHOT</version>
    <properties>
        <java.version>1.8</java.version>
        <!-- 确保这个 Flink 版本与你下载安装的版本一致 -->
        <flink.version>1.14.4</flink.version>
        <scala.binary.version>2.12</scala.binary.version>
        <maven.compiler.source>${java.version}</maven.compiler.source>
```

```
        <maven.compiler.target>${java.version}</maven.compiler.target>
    </properties>
    <dependencies>
        <!-- Flink Streaming Java API -->
        <dependency>
            <groupId>org.apache.flink</groupId>
            <artifactId>flink-streaming-java_${scala.binary.version}</artifa
ctId>
            <version>${flink.version}</version>
            <scope>provided</scope>
        </dependency>
        <!-- Flink Kafka Connector -->
        <dependency>
            <groupId>org.apache.flink</groupId>
            <artifactId>flink-connector-kafka_${scala.binary.version}</artif
actId>
            <version>${flink.version}</version>
        </dependency>
        <!-- Flink Client (for local execution and testing) -->
        <dependency>
            <groupId>org.apache.flink</groupId>
            <artifactId>flink-clients_${scala.binary.version}</artifactId>
            <version>${flink.version}</version>
            <scope>provided</scope>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <!-- Java Compiler Plugin -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.8.1</version>
                <configuration>
                    <source>${java.version}</source>
                    <target>${java.version}</target>
                </configuration>
            </plugin>
            <!-- Shade Plugin (to create an uber/fat JAR) -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-shade-plugin</artifactId>
                <version>3.2.4</version>
                <executions>
                    <execution>
                        <phase>package</phase>
```

```
                        <goals>
                            <goal>shade</goal>
                        </goals>
                        <configuration>
                            <artifactSet>
                                <excludes>
                                    <exclude>org.slf4j:slf4j-api</exclude>
                                    <exclude>log4j:log4j</exclude>
                                </excludes>
                            </artifactSet>
                            <filters>
                                <filter>
                                    <artifact>*:*</artifact>
                                    <excludes>
                                        <exclude>META-INF/*.SF</exclude>
                                        <exclude>META-INF/*.DSA</exclude>
                                        <exclude>META-INF/*.RSA</exclude>
                                    </excludes>
                                </filter>
                            </filters>
                            <transformers>
                                <transformer
implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTran
sformer">
                                    <!-- 指定 Main Class -->
                                    <mainClass>LogAlertingJob</mainClass>
                                </transformer>
                            </transformers>
                        </configuration>
                    </execution>
                </executions>
            </plugin>
        </plugins>
    </build>
</project>
```

log_producer.py

```
import time
import random
from kafka import KafkaProducer
import json

    # 初始化 Kafka 生产者
producer = KafkaProducer(
    bootstrap_servers=['localhost:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
```

```
)
    # 日志级别
log_levels = ['INFO', 'INFO', 'INFO', 'WARN', 'ERROR', 'DEBUG']
print("开始生成并发送日志...")
try:
    while True:
        log_message = {
            'timestamp': int(time.time() * 1000),
            'level': random.choice(log_levels),
            'message': f"This is a sample log message number {random.randint(1,
1000)}"
        }
            # 发送消息到 'log-stream' topic
        producer.send('log-stream', log_message)
        print(f"发送日志: {log_message}")
        time.sleep(random.uniform(0.5, 2)) # 随机间隔 0.5 到 2 秒
except KeyboardInterrupt:
    print("停止发送日志。")
finally:
    producer.close()
```

check_backpressure.sh

```
#!/bin/bash
JOB_ID=5198815b9a8153a9d9027d96f8d17fd5
if [ -z "$JOB_ID" ]; then
    echo "No running job found."
    exit 1
fi
echo "Checking backpressure for Job ID: $JOB_ID"
# 获取所有 vertex ID
VERTEX_IDS=$(curl -s http://localhost:8081/jobs/$JOB_ID | jq -r
'.vertices[].id')
while true; do
    echo "--- $(date) ---"
    for id in $VERTEX_IDS; do
        # 获取算子名称
        name=$(curl -s http://localhost:8081/jobs/$JOB_ID | jq -r --arg id "$id"
'.vertices[] | select(.id==$id) | .name')
        # 查询反压状态
        status=$(curl -s
http://localhost:8081/jobs/$JOB_ID/vertices/$id/backpressure | jq -r
'.status')
        echo "Vertex: $name | Status: $status"
    done
    sleep 5 # 每 5 秒检查一次
done
```